

CH-4 JAVASCRIPT

What is JavaScript?

1. JavaScript was designed to add interactivity to HTML pages
2. JavaScript is a scripting language (a scripting language is a lightweight programming language)
3. A JavaScript consists of lines of executable computer code
4. A JavaScript is usually embedded directly into HTML pages
5. JavaScript is an interpreted language (means that scripts execute without preliminary compilation)
6. Everyone can use JavaScript without purchasing a license

→ What can a JavaScript Do?

1. JavaScript gives HTML designers a programming tool - HTML authors are normally not programmers, but JavaScript is a scripting language with a very simple syntax! Almost anyone can put small "snippets" of code into their HTML pages
2. JavaScript can put dynamic text into an HTML page - A JavaScript statement like this: `document.write("<h1>" + name + "</h1>")` can write a variable text into an HTML page
3. JavaScript can react to events - A JavaScript can be set to execute when something happens, like when a page has finished loading or when a user clicks on an HTML element
4. JavaScript can read and write HTML elements - A JavaScript can read and change the content of an HTML element
5. JavaScript can be used to validate data - A JavaScript can be used to validate form data before it is submitted to a server, this will save the server from extra processing
6. JavaScript can be used to detect the visitor's browser - A JavaScript can be used to detect the visitor's browser, and - depending on the browser - load another page specifically designed for that browser
7. JavaScript can be used to create cookies - A JavaScript can be used to store and retrieve information on the visitor's computer

Insert Special Characters

The backslash (\) is used to insert apostrophes, new lines, quotes, and other special characters into a text string.

Look at the following JavaScript code:

```
var txt="We are the so-called "Vikings" from the north.";
document.write(txt);
```

In JavaScript, a string is started and stopped with either single or double quotes. This means that the string above will be chopped to: We are the so-called

To solve this problem, you must place a backslash (\) before each double quote in "Viking". This turns each double quote into a string literal:

```
var txt="We are the so-called \"Vikings\" from the north.";
document.write(txt);
```

JavaScript will now output the proper text string: We are the so-called "Vikings" from the north.

Here is another example:

```
document.write ("You \& I are singing!");
```

The example above will produce the following output:

```
You & I are singing!
```

The table below lists other special characters that can be added to a text string with the backslash sign:

Code	Outputs
\'	single quote
\"	double quote
\&	Ampersand

\\	Backslash
\n	new line
\r	carriage return
\t	Tab
\b	Backspace
\f	form feed

Some other important things to know when scripting with JavaScript.

JavaScript is Case Sensitive

A function named "myfunction" is not the same as "myFunction" and a variable named "myVar" is not the same as "myvar".

JavaScript is case sensitive - therefore watch your capitalization closely when you create or call variables, objects and functions.

White Space

JavaScript ignores extra spaces. You can add white space to your script to make it more readable. The following lines are equivalent:

```
name="Hege";  
name = "Hege";
```

Break up a Code Line

You can break up a code line **within a text string** with a backslash. The example below will be displayed properly:

```
document.write("Hello \  
World!");
```

However, you cannot break up a code line like this:

```
document.write \  
("Hello World!");
```

Basics of JAVASCRIPT

How to Put a JavaScript Into an HTML Page

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!");
</script>
</body>
</html>
```

The code above will produce this output on an HTML page:

Hello World!

Example Explained

To insert a JavaScript into an HTML page, we use the `<script>` tag. Inside the `<script>` tag we use the "type=" attribute to define the scripting language.

So, the `<script type="text/javascript">` and `</script>` tells where the JavaScript starts and ends:

```
<html>
<body>
<script type="text/javascript">
...
</script>
</body>
</html>
```

The word **document.write** is a standard JavaScript command for writing output to a page.

By entering the `document.write` command between the `<script>` and `</script>` tags, the browser will recognize it as a JavaScript command and execute the code line. In this case the browser will write Hello World! to the page:

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!");
</script>
</body>
</html>
```

Note: If we had not entered the <script> tag, the browser would have treated the document.write("Hello World!") command as pure text, and just write the entire line on the page.

HTML Comments to Handle Simple Browsers

Browsers that do not support JavaScript will display JavaScript as page content.

To prevent them from doing this, and as a part of the JavaScript standard, the HTML comment tag can be used to "hide" the JavaScript. Just add an HTML comment tag <!-- before the first JavaScript statement, and a --> (end of comment) after the last JavaScript statement.

```
<html>
<body>
<script type="text/javascript">
<!--
document.write("Hello World!");
//-->
</script>
</body>
</html>
```

The two forward slashes at the end of comment line (//) is the JavaScript comment symbol. This prevents JavaScript from executing the --> tag.

Where to Put the JavaScript

JavaScripts in a page will be executed immediately while the page loads into the browser. This is not always what we want. Sometimes we want to execute a script when a page loads, other times when a user triggers an event.

Scripts in the head section: Scripts to be executed when they are called, or when an event is triggered, go in the head section. When you place a script in the head section, you will ensure that the script is loaded before anyone uses it.

```
<html>
<head>
<script type="text/javascript">
....
</script>
</head>
```

Scripts in the body section: Scripts to be executed when the page loads go in the body section. When you place a script in the body section it generates the content of the page.

```
<html>
<head>
</head>
<body>
<script type="text/javascript">
....
</script>
</body>
```

Scripts in both the body and the head section: You can place an unlimited number of scripts in your document, so you can have scripts in both the body and the head section.

```
<html>
<head>
<script type="text/javascript">
....
```

```
</script>
</head>
<body>
<script type="text/javascript">
....
</script>
</body>
```

Using an External JavaScript

Sometimes you might want to run the same JavaScript on several pages, without having to write the same script on every page.

To simplify this, you can write a JavaScript in an external file. Save the external JavaScript file with a .js file extension.

Note: The external script cannot contain the <script> tag!

To use the external script, point to the .js file in the "src" attribute of the <script> tag:

```
<html>
<head>
<script src="xxx.js"></script>
</head>
<body>
</body>
</html>
```

Note: Remember to place the script exactly where you normally would write the script!

JavaScript Statements

A JavaScript statement is a command to the browser. The purpose of the command is to tell the browser what to do.

This JavaScript statement tells the browser to write "Hello Dolly" to the web page:

```
document.write("Hello Dolly");
```

It is normal to add a semicolon at the end of each executable statement. Most people think this is a good programming practice, and most often you will see this in JavaScript examples on the web.

The semicolon is optional (according to the JavaScript standard), and the browser is supposed to interpret the end of the line as the end of the statement. Because of this you will often see examples without the semicolon at the end.

Note: Using semicolons makes it possible to write multiple statements on one line.

JavaScript Code

JavaScript code (or just JavaScript) is a sequence of JavaScript statements.

Each statement is executed by the browser in the sequence they are written.

This example will write a header and two paragraphs to a web page:

```
<script type="text/javascript">
document.write("<h1>This is a header</h1>");
document.write("<p>This is a paragraph</p>");
document.write("<p>This is another paragraph</p>");
</script>
```

JavaScript Blocks

JavaScript statements can be grouped together in blocks.

Blocks start with a left curly bracket {, and ends with a right curly bracket }.

The purpose of a block is to make the sequence of statements execute together.

This example will write a header and two paragraphs to a web page:

```
<script type="text/javascript">
{
document.write("<h1>This is a header</h1>");
document.write("<p>This is a paragraph</p>");
document.write("<p>This is another paragraph</p>");
}
</script>
```

JavaScript Comments

Comments can be added to explain the JavaScript, or to make it more readable.

Single line comments start with `//`.

This example uses single line comments to explain the code:

```
<script type="text/javascript">
// This will write a header:
document.write("<h1>This is a header</h1>");
// This will write two paragraphs:
document.write("<p>This is a paragraph</p>");
document.write("<p>This is another paragraph</p>");
</script>
```

JavaScript Multi-Line Comments

Multi line comments start with `/*` and end with `*/`.

This example uses a multi line comment to explain the code:

```
<script type="text/javascript">
/*
The code below will write
one header and two paragraphs
*/
document.write("<h1>This is a header</h1>");
document.write("<p>This is a paragraph</p>");
document.write("<p>This is another paragraph</p>");
```

```
</script>
```

Using Comments to Prevent Execution

In this example the comment is used to prevent the execution of a single code line:

```
<script type="text/javascript">
document.write("<h1>This is a header</h1>");
document.write("<p>This is a paragraph</p>");
//document.write("<p>This is another paragraph</p>");
</script>
```

In this example the comments is used to prevent the execution of multiple code lines:

```
<script type="text/javascript">
/*
document.write("<h1>This is a header</h1>");
document.write("<p>This is a paragraph</p>");
document.write("<p>This is another paragraph</p>");
*/
</script>
```

Using Comments at the End of a Line

In this example the comment is placed at the end of a line:

```
<script type="text/javascript">
document.write("Hello"); // This will write "Hello"
document.write("Dolly"); // This will write "Dolly"
</script>
```

JavaScript Variables

As with algebra, JavaScript variables are used to hold values or expressions.

A variable can have a short name, like **x**, or a more describing name like **length**.

A JavaScript variable can also hold a text value like in **carname="Volvo"**.

Rules for JavaScript variable names:

- Variable names are case sensitive (**y** and **Y** are two different variables)
- Variable names must **begin with a letter** or the underscore character

NOTE: Because JavaScript is case-sensitive, variable names are case-sensitive.

Declaring (Creating) JavaScript Variables

Creating variables in JavaScript is most often referred to as "declaring" variables.

You can declare JavaScript variables with the **var statement**:

```
var x;  
var carname;
```

After the declaration shown above, the variables has no values, but you can assign values to the variables while you declare them:

```
var x=5;  
var carname="Volvo";
```

Note: When you assign a text value to a variable, you use quotes around the value.

Assigning Values to JavaScript Variables

You assign values to JavaScript variables with **assignment statements**:

```
x=5;  
carname="Volvo";
```

The variable name is on the left side of the = sign, and the value you want to assign to the variable is on the right.

After the execution of the statements above, the variable **x** will hold the value **5**, and **carname** will hold the value **Volvo**.

Assigning Values to Undeclared JavaScript Variables

If you assign values to variables that has not yet been declared, the variables will automatically be declared.

These statements:

```
x=5;  
carname="Volvo";
```

have the same effect as:

```
var x=5;  
var carname="Volvo";
```

Redeclaring JavaScript Variables

If you redeclare a JavaScript variable, it will not lose its original value.

```
var x=5;  
var x;
```

After the execution of the statements above, the variable **x** will still have the value of 5. The value of **x** is not reset (or cleared) when you redeclare it.

JavaScript Operators

The assignment operator = is used to assign values to JavaScript variables.

The arithmetic operator + is used to add values together.

```
y=5;
z=2;
x=y+z;
```

The value of x, after the execution of the statements above is 7.

Arithmetic Operators

Arithmetic operators are used to perform arithmetic between variables and/or values.

Given that **y=5**, the table below explains the arithmetic operators:

Operator	Description	Example	Result
+	Addition	x=y+2	x=7
-	Subtraction	x=y-2	x=3
*	Multiplication	x=y*2	x=10
/	Division	x=y/2	x=2.5
%	Modulus (division remainder)	x=y%2	x=1
++	Increment	x=++y	x=6
--	Decrement	x=--y	x=4

Assignment Operators

Assignment operators are used to assign values to JavaScript variables.

Given that **x=10** and **y=5**, the table below explains the assignment operators:

Operator	Example	Same As	Result
----------	---------	---------	--------

=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
=	x=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

Strings Operator (+)

The + operator can also be used to add string variables or text values together.

To add two or more string variables together, use the + operator.

```
txt1="What a very";
txt2="nice day";
txt3=txt1+txt2;
```

After the execution of the statements above, the variable txt3 contains "What a verynice day".

To add a space between the two strings, insert a space into one of the strings:

```
txt1="What a very ";
txt2="nice day";
txt3=txt1+txt2;
```

or insert a space into the expression:

```
txt1="What a very";
txt2="nice day";
txt3=txt1+" "+txt2;
```

After the execution of the statements above, the variable txt3 contains:

"What a very nice day"

Adding Strings and Numbers

Look at these examples:

```
x=5+5;
document.write(x);
```

```
x="5"+"5";
document.write(x);
```

```
x=5+"5";
document.write(x);
```

```
x="5"+5;
document.write(x);
```

The rule is:

If you add a number and a string, the result will be a string.

Comparison Operators (Relational Operator)

Comparison operators are used in logical statements to determine equality or difference between variables or values.

Given that **x=5**, the table below explains the comparison operators:

Operator	Description	Example
==	is equal to	x==8 is false
===	is exactly equal to (value and type)	x===5 is true x==="5" is false
!=	is not equal	x!=8 is true
>	is greater than	x>8 is false
<	is less than	x<8 is true
>=	is greater than or equal to	x>=8 is false
<=	is less than or equal to	x<=8 is true

How Can it be Used

Comparison operators can be used in conditional statements to compare values and take action depending on the result:

```
if (age<18) document.write("Too young");
```

Logical Operators

Logical operators are used to determine the logic between variables or values.

Given that **x=6 and y=3**, the table below explains the logical operators:

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
	or	(x==5 y==5) is false
!	not	!(x==y) is true

Conditional Operator (Ternary Operator ?:)

JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

Syntax

```
variablename=(condition)?value1:value2
```

Example

```
greeting=(visitor=="PRES")?"Dear President ":"Dear ";
```

If the variable **visitor** has the value of "PRES", then the variable **greeting** will be assigned the value "Dear President " else it will be assigned "Dear".

Conditional Structure & Looping Structure

Conditional statements in JavaScript are used to perform different actions based on different conditions.

Conditional Statements

Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- **if statement** - use this statement if you want to execute some code only if a specified condition is true
- **if...else statement** - use this statement if you want to execute some code if the condition is true and another code if the condition is false
- **if...else if...else statement** - use this statement if you want to select one of many blocks of code to be executed
- **switch statement** - use this statement if you want to select one of many blocks of code to be executed

If Statement

You should use the if statement if you want to execute some code only if a specified condition is true.

Syntax

```
if (condition)
{
  code to be executed if condition is true
}
```

Example 1

```
<script type="text/javascript">
Var str="";
if (str=="")
{
  alert("<b>Blank not allow</b>");
}
</script>
```

Note: **When comparing variables you must always use two equals signs next to each other (==)!**

Notice that there is no `..else..` in this syntax. You just tell the code to execute some code **only if the specified condition is true**.

If...else Statement

If you want to execute some code if a condition is true and another code if the condition is not true, use the `if....else` statement.

Syntax

```
if (condition)
{
code to be executed if condition is true
}
else
{
code to be executed if condition is not true
}
```

Example

```
<script type="text/javascript">
Var a=10;
Var b=5;

if (a<b)
{
document.write("a is minimum");
}
else
{
document.write("b is minimum");
}
</script>
```

If...else if...else Statement

You should use the if....else if...else statement if you want to select one of many sets of lines to execute.

Syntax

```
if (condition1)
{
  code to be executed if condition1 is true
}
else if (condition2)
{
  code to be executed if condition2 is true
}
else
{
  code to be executed if condition1 and
  condition2 are not true
}
```

Example

```
<script type="text/javascript">
Var a=10, b=5, c=8;
if (a<b && a<c)
{
  document.write("a is minimum");
}
Else if(b<a && b<c)
{
  document.write("b is minimum");
}
Else
{
  Document.write("c is minimum");
}
</script>
```

Switch Statement

You should use the switch statement if you want to select one of many blocks of code to be executed.

Syntax

```
switch(n)
{
case 1:
    execute code block 1
    break;
case 2:
    execute code block 2
    break;
default:
    code to be executed if n is
    different from case 1 and 2
}
```

This is how it works: First we have a single expression n (most often a variable), that is evaluated once. The value of the expression is then compared with the values for each case in the structure. If there is a match, the block of code associated with that case is executed. Use **break** to prevent the code from running into the next case automatically.

Example

```
<script type="text/javascript">
Day=1;
switch ()
{
case 1:
    document.write("Monday");
    break;
case 2:
    document.write("Tuesday");
    break;
default:
    document.write("I'm looking forward to this weekend!");
}</script>
```

Looping Structure

Loops in JavaScript are used to execute the same block of code a specified number of times or while a specified condition is true.

JavaScript Loops

Very often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal lines in a script we can use loops to perform a task like this.

In JavaScript there are two different kind of loops:

- **for** - loops through a block of code a specified number of times
- **while** - loops through a block of code while a specified condition is true

For Loop

The for loop is used when you know in advance how many times the script should run. The for statements has 3 parameters : i) initialize variables ii) condition iii) increment or decrement loop.

Syntax

```
for (var=startvalue;var<=endvalue;var=var+increment)
{
    code to be executed
}
```

Example

Explanation: The example below defines a loop that starts with i=0. The loop will continue to run as long as **i** is less than, or equal to 10. **i** will increase by 1 each time the loop runs.

```
<html>
<body>
<script type="text/javascript">
var i=0;
```

```
for (i=1;i<=3;i++)  
{  
document.write("The number is " + i);  
document.write("<br />");  
}  
</script>  
</body>  
</html>
```

Result

```
The number is 1  
The number is 2  
The number is 3
```

While Loop

- The while loop is used when you don't know how many times you have to loop, but you know you should stop loop when you meet the condition.
- The while loop is used when you want the loop to execute and continue executing while the specified condition is true.
- The value of expression is checked each time at the beginning of the loop, so even if this value changes during execution of the nested statements, execution will not stop until the end of the iteration.
- Sometimes, if the while expression evaluates to false from very beginning, the nested statements won't work even be run once.

Syntax

```
while (expression)  
{  
    code to be executed  
}
```

Example:

```
<html><body><script type="text/javascript">
```

```
var i=1;
while (i<=3){
document.write("The number is " + i);
document.write("<br />");
i=i+1;}
</script></body></html>
```

Result

The number is 1
The number is 2
The number is 3

The do...while Loop

- The do...while loop is a variant of the while loop.
- This loop will always execute a block of code atleast once.
- And then it will repeat the loop as long as the specified condition is true.
- This loop will always be executed at least once, even if the condition is false, because the code is executed before the condition is tested.**Syntax**

```
do
{
    code to be executed
}
while (var<=endvalue);
```

Example

```
<html>
<body>
<script type="text/javascript">
var i=0;
do
{
document.write("The number is " + i);
document.write("<br />");
i=i+1;
}
while (i<0);
```

```
</script>  
</body>  
</html>
```

Result

The number is 0

JavaScript Break and Continue

There are two special statements that can be used inside loops: break and continue.

Break

The break command will break the loop and continue executing the code that follows after the loop (if any).

Example

```
<html>  
<body>  
<script type="text/javascript">  
var i=0;  
for (i=0;i<=10;i++)  
{  
  if (i==3)  
  {  
    break;  
  }  
  document.write("The number is " + i);  
  document.write("<br />");  
}  
</script>  
</body>  
</html>
```

Result

The number is 0
The number is 1
The number is 2

Continue

The continue command will break the current loop and continue with the next value.

Example

```
<html>
<body>
<script type="text/javascript">
var i=0
for (i=0;i<=5;i++)
{
if (i==3)
{
continue;
}
document.write("The number is " + i);
document.write("<br />");
}
</script>
</body>
</html>
```

Result

The number is 0
The number is 1
The number is 2
The number is 4
The number is 5

JavaScript Dialog Boxes

In JavaScript we can create three kinds of popup or dialog boxes: Alert box, Confirm box, and Prompt box.

Alert Box

- An alert box is often used if you want to make sure information comes through to the user.
- When an alert box pops up, the user will have to click "OK" to proceed.
- Mostly it is used when user enters some invalid data or some warning is to be displayed.

Syntax:

```
alert("sometext");
```

Example :

```
<script>  
  alert("Hello! PGDCA Students....");  
</script>
```

Confirm Box

- A confirm box is often used if you want the user to verify or accept something.
- A confirm box displays the predefined message and "OK" and "Cancel" button.
- When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed.
- If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false.

Syntax:

```
confirm("sometext");
```

Example:

```
<script>  
    Confirm("Are you sure want change image??")  
</script>
```

Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

A prompt box display a predefined message, textbox for user input and "OK" and "Cancel" button.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

The prompt is of two parts: first is messages to user and second is default value, which will be returned on by pressing ok button without changing it.

Syntax:

```
prompt("sometext","defaultvalue");
```

Example:

```
<script>  
Nm=prompt("Enter your name");  
</script>
```

Array:

The Array object is used to store a set of values in a single variable name.

Defining Arrays:

The Array object is used to store a set of values in a single variable name.

We define an Array object with the new keyword. The following code line defines an Array object called myArray:

```
var myArray=new Array()
```

There are two ways of adding values to an array (you can add as many values as you need to define as many variables you require).

```
var mycars=new Array()  
mycars[0]="Saab"  
mycars[1]="Volvo"  
mycars[2]="BMW"
```

You could also pass an integer argument to control the array's size:

```
var mycars=new Array(3)  
mycars[0]="Saab"  
mycars[1]="Volvo"  
mycars[2]="BMW"
```

```
var mycars=new Array("Saab","Volvo","BMW")
```

Accessing Arrays:

You can refer to a particular element in an array by referring to the name of the array and the index number. The index number starts at 0.

The following code line:

```
Document.write(mycars[0])
```

will result in the following output:

```
Saab
```

Modify Values in Existing Arrays

To modify a value in an existing array, just add a new value to the array with a specified index number:

```
mycars[0]="Opel"
```

Now, the following code line:

```
Document.write(mycars[0])
```

will result in the following output:

```
Opel
```

User Defined Function :-

A function is a reusable code-block that will be executed by an event, or when the function is called.

JavaScript Functions

To keep the browser from executing a script when the page loads, you can put your script into a function.

A function contains code that will be executed by an event or by a call to that function.

You may call a function from anywhere within the page (or even from other pages if the function is embedded in an external .js file).

Functions can be defined both in the <head> and in the <body> section of a document. However, to assure that the function is read/loaded by the browser before it is called, it could be wise to put it in the <head> section.

Example

```
<html>
<head>
<script type="text/javascript">
function displaymessage()
{
alert("Hello World!");
}
</script>
</head>
<body>
<form>
```

```
<input type="button" value="Click me!"  
onclick="displaymessage()" >  
</form>  
</body>  
</html>
```

If the line: `alert("Hello world!!")` in the example above had not been put within a function, it would have been executed as soon as the line was loaded. Now, the script is not executed before the user hits the button. We have added an `onClick` event to the button that will execute the function `displaymessage()` when the button is clicked.

How to Define a Function

The syntax for creating a function is:

```
function functionname(var1,var2,...,varX)  
{  
  some code  
}
```

`var1, var2, etc` are variables or values passed into the function. The `{` and the `}` defines the start and end of the function.

```
function functionname()  
{  
  some code  
}
```

The return Statement

The return statement is used to specify the value that is returned from the function.

So, functions that are going to return a value must use the return statement.

Example

The function below should return the product of two numbers (a and b):

```
function prod(a,b)
{
x=a*b;
return x;
}
```

When you call the function above, you must pass along two parameters:

```
product=prod(2,3);
```

The returned value from the prod() function is 6, and it will be stored in the variable called product.

Built-in-Functions:

String:

charAt()

The charAt() method returns the **character at a specified position**.

Syntax:

```
stringObject.charAt(index)
```

Parameter	Description
index	Required. A number representing a position in the string

Example

```
<script type="text/javascript">
var str="Hello ";
document.write(str.charAt(2));
</script>
```

concat()

The concat() method is used to **join two or more strings**.

Syntax

```
stringObject.concat(stringX,stringX,...,stringX)
```

Parameter	Description
string	Required. One or more string objects to be joined to a string

Example

```
<script type="text/javascript">
var str1="Hello ";
var str2="world!";
document.write(str1.concat(str2));
</script>
```

indexOf()

The `indexOf()` method **returns the position of the first occurrence** of a specified string value in a string.

Syntax

```
stringObject.indexOf(searchvalue,fromindex)
```

Parameter	Description
searchvalue	Required. Specifies a string value to search for
fromindex	Optional. Specifies where to start the search

Example

```
<script type="text/javascript">
var str="Hello ";
document.write(str.indexOf("l"));
</script>
```

lastIndexOf()

The `lastIndexOf()` method returns the **position of the last occurrence** of a specified string value, searching backwards from the specified position in a string.

Syntax

```
stringObject.lastIndexOf(searchvalue,fromindex)
```

Parameter	Description
search	Required. Specifies a string value to search for

fromindex	Optional. Specifies where to start the search. Starting backwards in the string
-----------	---

Example

```
<script type="text/javascript">
var str="Hello ";
document.write(str.lastIndexOf("l"));
</script>
```

replace()

The replace() method is used to **replace some characters** with some other characters in a string.

Syntax

```
stringObject.replace(findstring,newstring)
```

Parameter	Description
findstring	Required. Specifies a string value to find. To perform a global search add a 'g' flag to this parameter and to perform a case-insensitive search add an 'i' flag
newstring	Required. Specifies the string to replace the found value from findstring

Example 1

```
<script type="text/javascript">
var str="Visit Microsoft!";
document.write(str.replace(/Microsoft/, "Google"));
</script>
```

The output of the code above will be:

```
Visit Google!
```

search()

The search() method is used to search a string for a specified value.

Syntax

```
stringObject.search(searchstring)
```

Parameter	Description
-----------	-------------

searchstring	Required. The value to search for in a string. To perform a case-insensitive search add an 'i' flag
--------------	---

Example 1

```
<script type="text/javascript">
var str="Visit Google!";
document.write(str.search(/Google /));
</script>
```

The output of the code above will be:

6

slice()

The slice() method **extracts a part of a string** and returns the extracted part in a new string.

Syntax

```
stringObject.slice(start,end)
```

Parameter	Description
start	Required. Specify where to start the selection. Must be a number. Starts at 0
end	Optional. Specify where to end the selection. Must be a number

Example

```
<script type="text/javascript">
var str="Hello happy world!";
document.write(str.slice(6));
</script>
```

The output of the code above will be:

happy world!

substr()

The substr() method **extracts a specified number of characters** in a string, from a start index.

Syntax

```
stringObject.substr(start,length)
```

Parameter	Description
start	Required. Where to start the extraction. Must be a numeric value
length	Optional. How many characters to extract. Must be a numeric value.

Example

```
<script type="text/javascript">
var str="Hello world!";
document.write(str.substr(3));
</script>
```

The output of the code above will be:

```
lo world!
```

substring()

The substring() method extracts the characters in a **string between two specified indices**.

Syntax

```
stringObject.substring(start,stop)
```

Parameter	Description
start	Required. Where to start the extraction. Must be a numeric value
stop	Optional. Where to stop the extraction. Must be a numeric value

Example

```
<script type="text/javascript">
var str="Hello world!";
document.write(str.substring(3,7));
</script>
```

The output of the code above will be:

```
lo w
```

toLowerCase()

The toLowerCase() method is used to **display a string in lowercase** letters.

Syntax

```
stringObject.toLowerCase()
```

Example

```
<script type="text/javascript">
var str="Hello world!";
document.write(str.toLowerCase());
</script>
```

toUpperCase()

The toUpperCase() method is used to **display a string in uppercase** letters.

Syntax

```
stringObject.toUpperCase()
```

Example

```
<script type="text/javascript">
var str="Hello world!";
document.write(str.toUpperCase());
</script>
```

String Object Property

Length

The length property returns the **number of characters in a string**.

Syntax

```
stringObject.length
```

Example

```
<script type="text/javascript">
var txt="Hello World!";
document.write(txt.length);
</script>
```

The output of the code above will be:

12

Date:

- 1. Date:** The Date() method returns today's date and time.

Syntax:

Date()

Example:

```
<script type="text/javascript">
    document.write(Date())
</script>
```

- 2. getDate:** The getDate() method returns the day of the month.(1 to 31)

Syntax:

dateObject.getDate()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getDate())
</script>
```

- 3. getDay:** The getDay() method returns a number that represents the day of the week.(0 to 6 starting from sunday)

Syntax: dateObject.getDay()

Example:

```
<script type="text/javascript">
    var d=new Date()
    document.write(d.getDay())
</script>
```

3. getMonth: The getMonth() method returns the month, as a number.(0 to 11 from January).

Syntax:

dateObject.getMonth()

Example:

```
<script type="text/javascript">
    var d=new Date()
    document.write(d.getMonth())
</script>
```

4. getFullYear: The getFullYear() method returns a four-digit number that represents a year.

Syntax: dateObject.getFullYear()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getFullYear())
</script>
```

5. getyear: The getYear() method returns the year, as a two-digit OR a four-digit number.

Syntax: dateObject.getYear()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getYear())
</script>
```

6. getHours: The getHours() method returns the hour of a time.

Syntax: dateObject.getHours()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getHours())
</script>
```

7. getMinutes: The getMinutes() method returns the minutes of a time.

Syntax: dateObject.getMinutes()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getMinutes())
</script>
```

8. getSeconds: The getSeconds() method returns the seconds of a time.

Syntax: dateObject.getSeconds()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getSeconds())
</script>
```

9. getMilliseconds: The getMilliseconds() method returns the milliseconds of a time.

Syntax: dateObject.getMilliseconds()

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.getMilliseconds())
</script>
```

10. setDate: The setDate() method is used to set the day of the month.

Syntax: dateObject.setDate(day)

day: Required. A numeric value (from 1 to 31) that represents a day in a month

Example:

```
<script type="text/javascript">
    var d = new Date()
    d.setDate(15)
    document.write(d)
</script>
```

11. setMonth: The setMonth() method is used to set the month.

Syntax: dateObject.setMonth(month,day)

month: A numeric value between 0 and 11 representing the month

day: A numeric value between 1 and 31 representing the day

Example:

```
<script type="text/javascript">
    var d=new Date()
    d.setMonth(0)
    document.write(d)
</script>
```

12. setYear: The setYear() method is used to set the year.

Syntax: dateObject.setYear(year)

year: A two or four digit number that indicates the year

Example:

```
<script type="text/javascript">
    var d = new Date()
    d.setYear(1891)
    document.write(d)
</script>
```

13. setFullYear: The setFullYear() method is used to set the year.

Syntax: dateObject.setFullYear(year,month,day)

Example:

year: A four-digit value representing the year
 month: A numeric value between 0 and 11 representing the month
 day: A numeric value between 1 and 31 representing the date

Example:

```
<script type="text/javascript">
    var d = new Date()
    d.setFullYear(1992)
    document.write(d)
</script>
```

13. setHours: The setHours() method is used to set the hour of a specified time.

Syntax: dateObject.setHours(hour,min,sec,millisec)

hour: A numeric value between 0 and 23 representing the hour

min: A numeric value between 0 and 59 representing the minutes

sec: A numeric value between 0 and 59 representing the seconds

millisec: A numeric value between 0 and 999 representing the milliseconds

Example:

```
<script type="text/javascript">
```

```

    var d = new Date()
    d.setHours(15)
    document.write(d)
</script>

```

- 14. setMinutes:** The setMinutes() method is used to set the minutes of a specified time.

Syntax: dateObject.setMinutes(min,sec,millisec)

min: A numeric value between 0 and 59 representing the minutes

sec: A numeric value between 0 and 59 representing the seconds

millisec: A numeric value between 0 and 999 representing the milliseconds

Example:

```

<script type="text/javascript">
    var d = new Date()
    d.setMinutes(1)
    document.write(d)
</script>

```

- 15. setSeconds:** The setSeconds() method is used to set the seconds of a specified time.

Syntax: dateObject.setSeconds(sec,millisec)

sec: A numeric value between 0 and 59 representing the seconds

millisec: A numeric value between 0 and 999 representing the milliseconds

Example:

```

<script type="text/javascript">
    var d = new Date()
    d.setSeconds(1)
    document.write(d)
</script>

```

- 16. setDay:** The setDay() method returns a number that represents the day of the week.(0 to 6 starting from sunday)

Syntax: dateObject.setDay(day)

Example:

```

<script type="text/javascript">
    var d=new Date()

```

```
document.write(d.setDay(2))
</script>
```

17. setMilliseconds: The setMilliseconds() method returns the milliseconds of a time.

Syntax: dateObject.setMilliseconds(milliseconds)

Example:

```
<script type="text/javascript">
    var d = new Date()
    document.write(d.setMilliseconds(60))
</script>
```

Math Function:

- Another way of performing a calculation is to use the JavaScript built-in Math methods.
- These methods are applied to an object called the Math object.
- The syntax for applying a Math method is:

value = Math.method(variable);

- For example,

AbsValue = Math.abs(NumVar);

1. abs: The abs() method returns the **absolute value of a number**.

Syntax: Math.abs(x)

x: Must be a numeric value

Example:

```
<script type="text/javascript">
    document.write(Math.abs(7.25) + "<br />")
    document.write(Math.abs(-7.25) + "<br />")
    document.write(Math.abs(7.25-10))
</script>
```

2. ceil: The `ceil()` method returns the value of a number rounded UPWARDS to the nearest integer.

Syntax: `Math.ceil(x)`

x: Must be a numeric value

Example:

```
<script type="text/javascript">
    document.write(Math.ceil(0.60) + "<br />")
    document.write(Math.ceil(0.40) + "<br />")
    document.write(Math.ceil(5) + "<br />")
    document.write(Math.ceil(5.1) + "<br />")
    document.write(Math.ceil(-5.1) + "<br />")
    document.write(Math.ceil(-5.9))
</script>
```

3. floor: The `floor()` method returns the value of a number rounded DOWNWARDS to the nearest integer.

Syntax: `Math.floor(x)`

x: Must be an number.

Example:

```
<script type="text/javascript">
    document.write(Math.floor(0.60) + "<br />")
    document.write(Math.floor(0.40) + "<br />")
    document.write(Math.floor(5) + "<br />")
    document.write(Math.floor(5.1) + "<br />")
    document.write(Math.floor(-5.1) + "<br />")
    document.write(Math.floor(-5.9))
</script>
```

4. pow: The `pow()` method returns the value of x to the power of y (x^y).

Syntax: `Math.pow(x,y)`

x: A number

y: A number

Example:

```
<script type="text/javascript">
    document.write(Math.pow(0,0) + "<br />")
    document.write(Math.pow(0,1) + "<br />")
    document.write(Math.pow(1,1) + "<br />")
    document.write(Math.pow(1,10) + "<br />")
    document.write(Math.pow(2,3) + "<br />")
    document.write(Math.pow(-2,3) + "<br />")
    document.write(Math.pow(2,4) + "<br />")
    document.write(Math.pow(-2,4) + "<br />")
</script>
```

5. random: The random() method returns a random number between 0 and 1.

Syntax: Math.random()

Example:

```
<script type="text/javascript">
    document.write(Math.random())
</script>
```

6. round: The round() method rounds a number to the nearest integer.

Syntax: Math.round(x)

x: A number

Example:

```
<script type="text/javascript">
    document.write(Math.round(0.60) + "<br />")
    document.write(Math.round(0.50) + "<br />")
    document.write(Math.round(0.49) + "<br />")
    document.write(Math.round(-4.40) + "<br />")
    document.write(Math.round(-4.60))
</script>
```

7. max: The max() method returns the number with the highest value of two specified numbers.

Syntax: Math.max(x,y)

x: A number
y: A number

Example:

```
<script type="text/javascript">
    document.write(Math.max(5,7) + "<br />")
    document.write(Math.max(-3,5) + "<br />")
    document.write(Math.max(-3,-5) + "<br />")
    document.write(Math.max(7.25,7.30))
</script>
```

8. min: The min() method returns the number with the lowest value of two specified numbers.

Syntax: Math.min(x,y)

x: A number
y: A number

Example:

```
<script type="text/javascript">
    document.write(Math.min(5,7) + "<br />")
    document.write(Math.min(-3,5) + "<br />")
    document.write(Math.min(-3,-5) + "<br />")
    document.write(Math.min(7.25,7.30))
</script>
```

Array Function :-

The Array object is used to store multiple values in a single variable.

Property:-

[length](#) Sets or returns the number of elements in an array

Methods:-

1. Join: The join() method is used to put all the elements of an array into a string. The elements will be separated by a specified separator.

Syntax: `arrayObject.join(separator)`
separator: Specifies the separator to be used.

Example:

```
<script type="text/javascript">
    var arr = new Array(3)
    arr[0] = "Nikita"
    arr[1] = "Devhuti"
    arr[2] = "Dimple"
    document.write(arr.join() + "<br />")
    document.write(arr.join("."))
</script>
```

2. reverse: The `reverse()` method is used to reverse the order of the elements in an array.

Syntax: `arrayObject.reverse()`

Example:

```
<script type="text/javascript">
    var arr = new Array(3)
    arr[0] = "Nikita"
    arr[1] = "Devhuti"
    arr[2] = "Dimple"
    document.write(arr + "<br />")
    document.write(arr.reverse())
</script>
```

2. pop: The `pop()` method is used to remove and return the last element of an array.

Syntax: `arrayObject.pop()`

Example:

```
<script type="text/javascript">
    var arr = new Array(3)
    arr[0] = "Nikita"
    arr[1] = "Devhuti"
```

```

arr[2] = "Dimple"
document.write(arr + "<br />")
document.write(arr.pop() + "<br />")
document.write(arr)
</script>

```

- 3. push:** The push() method adds one or more elements to the end of an array and returns the new length.

Syntax: arrayObject.push(newelement1,newelement2,.....,newelementX)

newelement1: The first element to add to the array.

newelement2: The second element to add to the array.

Example:

```

<script type="text/javascript">
var arr = new Array(3)
arr[0] = "Nikita"
arr[1] = "Devhuti"
arr[2] = "Dimple"
document.write(arr + "<br />")
document.write(arr.push("Jags") + "<br />")
document.write(arr)
</script>

```

- 4. shift:** The shift() method is used to remove and return the first element of an array.

Syntax: arrayObject.shift()

Example:

```

<script type="text/javascript">
var arr = new Array(3)
arr[0] = "Nikita"
arr[1] = "Devhuti"
arr[2] = "Dimple"
document.write(arr + "<br />")
document.write(arr.shift() + "<br />")
document.write(arr)

```

</script>

5. sort: The sort() method is used to sort the elements of an array.

Syntax: arrayObject.sort(sortby)

sortby: Specifies the sort order. Must be a function

Example:

```
<script type="text/javascript">
  var arr = new Array(6)
  arr[0] = "Jani"
  arr[1] = "Hege"
  arr[2] = "Stale"
  arr[3] = "Kai Jim"
  arr[4] = "Borge"
  arr[5] = "Tove"
  document.write(arr + "<br />")
  document.write(arr.sort())
</script>
```

Document Object

The Document object represents the entire HTML document and can be used to access all elements in a page.

The Document object is part of the Window object and is accessed through the window.document property.

HTML DOM forms Collection

The forms collection returns a reference to all Form objects in the document.

Syntax

```
document.forms[]
```

Example

```
<html>
<body>
<form name="Form1"></form>
<form name="Form2"></form>
<form name="Form3"></form>
<script type="text/javascript">
document.write("This document contains: ")
document.write(document.forms.length + " forms.")
</script>
</body>
</html>
```

HTML DOM write() Method

The write() method writes HTML expressions or JavaScript code to a document.

Note: Multiple arguments (exp1,exp2,exp3,...) can be listed and they will be appended to the document in order of occurrence.

Syntax

```
document.write(exp1,exp2,exp3,...)
```

Example

```
<html>
<body>
<script type="text/javascript">
document.write("Hello World!")
</script>
</body>
</html>
```

HTML DOM `getElementsByName()` Method

The `getElementsByName()` method returns a collection of objects with the specified NAME.

Syntax

```
document.getElementsByName(name)
```

Example

```
<html>
<head>
<script type="text/javascript">
function getElements()
{
var x=document.getElementsByName("myInput");
alert(x.length);
}
</script>
</head>
<body>
<input name="myInput" type="text" size="20" /><br />
<input name="myInput" type="text" size="20" /><br />
<input name="myInput" type="text" size="20" /><br />
<br />
```

```
<input type="button" onclick="getElements()"
value="How many elements named 'myInput'?" />
</body>
</html>
```

HTML DOM getElementById() Method

The getElementById() method returns a reference to the first object with the specified ID.

Syntax

```
document.getElementById(id)
```

innerText:- innerText sets or gets all of the content within a given element.

innerHTML:- innerHTML sets or gets all of the markup and content within a given element.

Example

```
<html>
<head>
<script type="text/javascript">
function getValue()
{
  var x=document.getElementById("myHeader")
  alert(x.innerHTML)
}
</script>
</head>
<body>
<h1 id="myHeader" onclick="getValue()">This is a header</h1>
<p>Click on the header to alert its value</p>
</body>
</html>
```

Navigator Object

- The Navigator object is actually a JavaScript object, not an HTML DOM object.

- The Navigator object is automatically created by the JavaScript runtime engine and contains information about the client browser.

Navigator Object Collections:-

- `plugins[]`: Returns a reference to all embedded objects in the document

Navigator Object Properties:-

Property	Description
• <code>appName</code>	• Returns the code name of the browser
• <code>appMinorVersion</code>	• Returns the minor version of the browser
• <code>appName</code>	• Returns the name of the browser
• <code>appVersion</code>	• Returns the platform and version of the browser
• <code>browserLanguage</code>	• Returns the current browser language
• <code>cookieEnabled</code>	• Returns a Boolean value that specifies whether cookies are enabled in the browser
• <code>cpuClass</code>	• Returns the CPU class of the browser's system
• <code>onLine</code>	• Returns a Boolean value that specifies whether the system is in offline mode
• <code>platform</code>	• Returns the operating system platform
• <code>systemLanguage</code>	• Returns the default language used by the OS
• <code>userAgent</code>	• Returns the value of the user-agent header sent by the client to the server
• <code>userLanguage</code>	• Returns the OS' natural language setting

Navigator Object Methods:-

Method	Description
• <code>javaEnabled()</code>	• Specifies whether or not the browser has Java enabled
• <code>taintEnabled()</code>	• Specifies whether or not the browser has data tainting enabled

Example:-

```
<script type="text/javascript">
var x = navigator
document.write("CodeName=" + x.appCodeName)
document.write("<br />")
document.write("MinorVersion=" + x.appMinorVersion)
document.write("<br />")
document.write("Name=" + x.appName)
document.write("<br />")
document.write("Version=" + x.appVersion)
document.write("<br />")
document.write("CookieEnabled=" + x.cookieEnabled)
document.write("<br />")
document.write("CPUClass=" + x.cpuClass)
document.write("<br />")
document.write("OnLine=" + x.onLine)
document.write("<br />")
document.write("Platform=" + x.platform)
document.write("<br />")
document.write("UA=" + x.userAgent)
document.write("<br />")
document.write("BrowserLanguage=" + x.browserLanguage)
document.write("<br />")
document.write("SystemLanguage=" + x.systemLanguage)
document.write("<br />")
document.write("UserLanguage=" + x.userLanguage)
</script>
```

History Object

- The History object is actually a JavaScript object, not an HTML DOM object.
- The History object is automatically created by the JavaScript runtime engine and consists of an array of URLs. These URLs are the URLs the user has visited within a browser window.
- The History object is part of the Window object and is accessed through the window.history property.

History Object Properties:-

Length:-

The length property returns the number of elements in the history list.

Syntax

history.length

Example

```
<script type="text/javascript">
document.write(history.length);
</script>
```

History Object Methods:-

Method	Description
back()	Loads the previous URL in the history list
forward()	Loads the next URL in the history list
go()	Loads a specific page in the history list

Back():-

The back() method loads the previous URL in the history list. This is the same as clicking the Back button and history.go(-1).

Syntax

history.back()

Example

The following example creates a back-button on a page:

```
<html>
<head>
<script type="text/javascript">
function goBack()
{
    window.history.back()
}
</script>
</head>
<body>
<input type="button" value="Back" onclick="goBack()" />
</body>
</html>
```

Forward():-

The forward() method loads the next URL in the history list.

This is the same as clicking the Forward button and history.go(1).

Syntax

```
history.forward()
```

Example

The following example creates a forward-button on a page:

```
<html>
<head>
<script type="text/javascript">
function goForward()
{
    window.history.forward()
}
</script>
</head>
<body>
<input type="button" value="Forward" onclick="goForward()" />
</body>
</html>
```

Go():-

The go() method loads a specific page in the history list.

Syntax

```
history.go(number|URL)
```

Example

The following example loads the previous page in the history list:

```
<html>
<head>
<script type="text/javascript">
function goBack()
{
```

```

window.history.go(-1)
}
</script>
</head>
<body>
<input type="button" value="Back" onclick="goBack()" />
</body>
</html>

```

Location Object

The location object contains information about the current URL.

The location object is part of the window object and is accessed through the window.location property.

Note: There is no public standard that applies to the location object, but all major browsers support it.

Location Object Properties

Property	Description
hash	Sets or returns the anchor portion of a URL
host	Sets or returns the hostname and port of a URL
hostname	Sets or returns the hostname of a URL
href	Sets or returns the entire URL
pathname	Sets or returns the path name of a URL
port	Sets or returns the port number the server uses for a URL
protocol	Sets or returns the protocol of a URL
search	Sets or returns the query portion of a URL

Location Object Methods

Method	Description
assign()	Loads a new document
reload()	Reloads the current document

[replace\(\)](#) Replaces the current document with a new one

User define object:-

- Javascript permit the creation of user define object.
- A user define object will also be associated with properties and methods.
- After creation of such an object, any number of instances of this object can be created & used.
- Let's illustrate with an example: A person is an object.
- Properties are the values associated with the object. The person's
- properties include name, height, weight, age, skin, tone, eyecolor etc.
- All person have these properties, but the values of those properties will differ from person to person.
- Object also have methods. Methods are the actions that can be performed on object.
- The persons method can be eat(),sleep(),work(),play() etc.

There are two different ways to create a new object :

[1]Create a direct instance of an object :

The following code creates an instance of an object and adds four properties to it.

```
personObj=new Object()
personObj.firstname='madhu'
personObj.lastname='rushabh'
personObj.age=21
personObj.eyecolor='blue'
```

[2]Create template on an object :

The template defines the structure of an object:

```
Function person(fname,lname,age,eyecolor)
{
This.firstname=fname;
This.lastname=lname;
This.age=age;
This.eyecolor=eyecolor;
}
```

This template is just a function. Inside the function you need to assign things to this. Property name.

Here “this” refer to the current object in focus.

For example, this.age will refer to the age of the current object.

Once you have created template, you can create new instances of object, like this.

Events:-

1.onabort:-

?? This is event occurs when loading of an image is aborted.

?? This event is support only on image object

2.onblur:-

?? This is event occurs when an object loses focus.

?? This event is supports:- button, checkbox, frame, password, radio, submit, textarea, window

3.onchange:-

?? This is event occurs when the content of a field changes.

?? This event is supports by following objects:- text, textarea, select

4.onclick:-

?? This is event occurs when an object gets clicked.

?? This event is supports by following object :button, document, checkbox, link, radio, reset, submit.

Example:

```
<html>
<head>
<script type="text/javascript">
<!--
function sayHello() {
    alert("Hello World")
}
//-->
</script>
</head>
<body>
<input type="button" onclick="sayHello()" value="Say Hello" />
```

</body>

</html>

5.ondblclick:-

?? This is event occurs when object gets double clicked.

?? This event is support only on document,link

6.onerror:-

?? This is event is triggered when an error occurs loading a document or an image.

?? This event is support only on image,window object

7.onfocus:-

?? This is event occurs when an object gets focus.

?? This event is supports by following objects:- button, checkbox, radio, frame, password, reset, submit, text, textarea, window.

8.onkeydown:-

?? This is event occurs when keyboard key is pressed.

?? This event is supports by following objects:- image, link, document, textarea.

9.onkeypress:-

?? This is event occurs when keyboard key is pressed or held down.

?? This event is supports by following objects:- image, link, document, textarea.

10.onkeyup:-

?? This is event occurs when keyboard key is released.

?? This event is supports by following objects:- image, link, document, textarea.

11.onmousedown:-

?? This is event occurs when a mouse button is clicked or down.

?? This event is supports by following objects:- button, link, document.

12.onmouseup:-

?? This is event occurs when a mouse button is released or up.

?? This event is supports by following objects:- button, link, document.

13.onmouseover:-

?? This is event occurs when a mouse the mouse pointer moves over a specified object.

?? This event is supports by following objects:- button, link, document.

Example:

```
<html>
<head>
<script type="text/javascript">
<!--
function over() {
    alert("Mouse Over");
}
//-->
</script>
</head>
<body>
<div onmouseover="over()">
<h2> This is inside the division </h2>
</div>
</body>
</html>
```

14.onmousemove:-

?? This is event occurs when a mouse pointer is moved.

?? This event is supports by following objects:- button, link, document.

15.onmouseout:-

?? This is event occurs when a mouse pointer moves away from the specified object.

?? This event is supports by following objects:- button, link, document.

Example

```
<html>
<head>
<script type="text/javascript">
<!--
function out() {
    alert("Mouse Out");
}
//-->
```

```
//-->
</script>
</head>
<body>
<div onmouseout="out()">
<h2> This is inside the division </h2>
</div>
</body>
</html>
```

16.onselect:-

?? This is event occurs when a text is selected in text or in textarea

?? This event is supports by following objects:- text, textarea.

17.onsubmit:-

?? This is event occurs when a submit button in a form is clicked.

?? This event is supports by following objects:- form

Example:

```
<html>
<head>
<script type="text/javascript">
<!--
function validation() {
    all validation goes here
    .....
    return either true or false
}
//-->
</script>
</head>
<body>
<form method="POST" action="t.cgi" onsubmit="return validate()">
.....
<input type="submit" value="Submit" />
</form>
</body>
</html>
```

18.onreset:-

?? This is event occurs when a reset button in a form is clicked.

?? This event is supports by following objects:- form

20.onload:-

?? This is event occurs immediately after a page or an image is loaded.

?? This event is supports by following objects:- image, window, layer.

21.onunload:-

?? This is event occurs when a user exits a page.

?? This event is supports by following HTML tags:- <body> <frameset>

?? This event is supports by following objects:- window.

22. Timer Event:-

With JavaScript, it is possible to execute some code at specified time-intervals. This is called timing events.

It's very easy to time events in JavaScript. The two key methods that are used:

- setInterval() - executes a function, over and over again, at specified time intervals
- setTimeout() - executes a function, once, after waiting a specified number of milliseconds

setTimeout()

Syntax

```
window.setTimeout("javascript function",milliseconds);
```

The **window.setTimeout()** method can be written without the window prefix.

The setTimeout() method will wait the specified number of milliseconds, and then execute the specified function.

The first parameter of setTimeout() should be a function.

The second parameter indicates how many milliseconds, from now, you want to execute the first parameter.

Example

Wait 3 seconds, then alert "Hello":

```
setTimeout(function(){alert("Hello")},3000);
```

How to Stop the Execution?

The `clearTimeout()` method is used to stop the execution of the function specified in the `setTimeout()` method.

Syntax

`window.clearTimeout(timeoutVariable)`

The **`window.clearTimeout()`** method can be written without the window prefix.

To be able to use the `clearTimeout()` method, you must use a global variable when creating the timeout method:

```
myVar=setTimeout("javascript function",milliseconds);
```

Then, if the function has not already been executed, you will be able to stop the execution by calling the `clearTimeout()` method.

Example

Same example as above, but we have added a "Stop the alert" button:

```
var myVar;  
function myFunction()  
{  
myVar=setTimeout(function(){alert("Hello")},3000);  
}  
function myStopFunction()  
{  
clearTimeout(myVar);  
}
```

Cookie:-

- A cookie is often used to **identify a user**.
- A cookie is a variable that is stored on the visitor's computer.
- Each time the same computer requests a page with a browser, it will send the cookie too.
- With JavaScript, you can both create and retrieve cookie values.

Cookies are a plain text data record of 5 variable-length fields:

Expires : The date the cookie will expire. If this is blank, the cookie will expire when the visitor quits the browser.

Domain : The domain name of your site.

Path : The path to the directory or web page that set the cookie. This may be blank if you want to retrieve the cookie from any directory or page.

Secure : If this field contains the word "secure" then the cookie may only be retrieved with a secure server. If this field is blank, no such restriction exists.

Name=Value : Cookies are set and retrieved in the form of key and value pairs.

Creating a Cookie:

The simplest way to create a cookie is to assign a string value to the *document.cookie* object, which looks like this:

Syntax:-

`document.cookie = "name1=value1;name2=value2;expires=date";`

Here *expires* attribute is option. If you provide this attribute with a valid date or time then cookie will expire at the given date or time and after that cookies' value will not be accessible.

Reading Cookies:

Reading a cookie is just as simple as writing one, because the value of the *document.cookie* object is the cookie. So you can use this string whenever you want to access the cookie.

The *document.cookie* string will keep a list of *name=value* pairs separated by semicolons, where *name* is the *name* of a cookie and *value* is its string value.

Example:

Deleting a Cookie:

Sometimes you will want to delete a cookie so that subsequent attempts to read the cookie return nothing. To do this, you just need to set the expiration date to a time in the past.

Example:

The following example illustrates how to delete cookie by setting expiration date one Month in past :

```
<html>
<head>
<script type="text/javascript">
<!--
function WriteCookie()
{
    var now = new Date();
    now.setMonth( now.getMonth() - 1 );
    cookievalue = escape(document.myform.customer.value) + ";";
    document.cookie="name=" + cookievalue;
    document.cookie = "expires=" + now.toUTCString() + ";";
    alert("Setting Cookies : " + "name=" + cookievalue );
}
//-->
</script>
</head>
<body>
```

```
<form name="formname" action="">
Enter name: <input type="text" name="customer"/>
<input type="button" value="Set Cookie" onclick="WriteCookie()"/>
</form>
</body>
</html>
```

HTML Form Validation

What is form validation?

Form validation is the process of making sure that data supplied by the user using a form, meets the criteria set for collecting data from the user. For example, if you are using a registration form, and you want your user to submit name, email id and address, you must use a code (in JavaScript or in any other language) to check whether the user entered a name containing alphabets only, a valid email address and a proper address.

JavaScript Form Validation Example

In this example, we are going to validate the name and password. The name can't be empty and password can't be less than 6 characters long.

Here, we are validating the form on form submit. The user will not be forwarded to the next page until given values are correct.

```
1. <script>
2. function validateform(){
3.   var name=document.myform.name.value;
4.   var password=document.myform.password.value;
5.
6.   if (name==null || name==""){
7.     alert("Name can't be blank");
8.     return false;
9.   }else if(password.length<6){
10.     alert("Password must be at least 6 characters long.");
11.     return false;
12.   }
13. }
```

```

14.     </script>
15.     <body>
16.     <form name="myform" method="post" action="abc.jsp" onsubmit
      ="return validateform()" >
17.       Name: <input type="text" name="name"><br/>
18.       Password: <input type="password" name="password"><br/>
19.       <input type="submit" value="register">
20.     </form>

```

JavaScript Retype Password Validation

```

1. <script type="text/javascript">
2. function matchpass(){
3.   var firstpassword=document.f1.password.value;
4.   var secondpassword=document.f1.password2.value;
5.
6.   if(firstpassword==secondpassword){
7.     return true;
8.   }
9.   else{
10.    alert("password must be same!");
11.    return false;
12.   }
13.   }
14. </script>
15.
16. <form name="f1" action="register.jsp" onsubmit="return matchp
    ass()">
17.   Password:<input type="password" name="password" /><br/>
18.   Re-
    enter Password:<input type="password" name="password2"/><br/>
19.   <input type="submit">
20. </form>

```

JavaScript Number Validation

```

1. <script>
2. function validate(){
3.   var num=document.myform.num.value;

```

```

4. if (isNaN(num)){
5.   document.getElementById("numloc").innerHTML="Enter Numeric val
   ue only";
6.   return false;
7. }else{
8.   return true;
9. }
10.  }
11.   </script>
12.   <form name="myform" onsubmit="return validate()" >
13.     Number: <input type="text" name="num"><span id="numloc"></
     span><br/>
14.     <input type="submit" value="submit">
15.   </form>

```

JavaScript validation with image

Let's see an interactive JavaScript form validation example that displays correct and incorrect image if input is correct or incorrect.

```

1. <script>
2. function validate(){
3.   var name=document.f1.name.value;
4.   var password=document.f1.password.value;
5.   var status=false;
6.
7.   if(name.length<1){
8.     document.getElementById("nameloc").innerHTML=
9.     " <img src='unchecked.gif' /> Please enter your name";
10.    status=false;
11.   }else{
12.     document.getElementById("nameloc").innerHTML=" <img src='ch
     ecked.gif' />";
13.    status=true;
14.   }
15.   if(password.length<6){
16.     document.getElementById("passwordloc").innerHTML=
17.     " <img src='unchecked.gif' /> Password must be at least 6 char lon
     g";

```

```

18.     status=false;
19.     }else{
20.     document.getElementById("passwordloc").innerHTML=" <img sr
      c='checked.gif' />";
21.     }
22.     return status;
23.     }
24.     </script>
25.
26.     <form name="f1" action="#" onsubmit="return validate()">
27.     <table>
28.     <tr><td>Enter Name:</td><td><input type="text" name="name"
      />
29.     <span id="nameloc"></span></td></tr>
30.     <tr><td>Enter Password:</td><td><input type="password" nam
      e="password" />
31.     <span id="passwordloc"></span></td></tr>
32.     <tr><td colspan="2"><input type="submit" value="register" /></t
      d></tr>
33.     </table>
34.     </form>

```

JavaScript email validation

We can validate the email by the help of JavaScript.

There are many criteria that need to be follow to validate the email id such as:

- email id must contain the @ and . character
- There must be at least one character before and after the @.
- There must be at least two characters after . (dot).
-

```

<script>
function validateemail()
{
var x=document.myform.email.value;
var atposition=x.indexOf("@");
var dotposition=x.lastIndexOf(".");
if (atposition<1 || dotposition<atposition+2 || dotposition+2>=x.length){

```

```
    alert("Please enter a valid e-  
mail address \n atpostion:"+atposition+"\n dotposition:"+dotposition);  
    return false;  
  }  
}  
</script>  
<body>  
<form name="myform" method="post" action="#" onsubmit="return valid  
ateemail();">  
Email: <input type="text" name="email"><br/>  
  
<input type="submit" value="register">  
</form>
```